generate a URL-encoded string of values suitable for posting easily. This interface is not capable of serializing objects that have nested properties.

11.2.3 Working with JsonResult

Up to now, you have passed simple values and JavaScript objects to the page handler. This section will begin to look at working with JSON, which has become the de facto data format for exchanging data between the client and the server in web applications. You will use plain JavaScript for this example and serialize the JavaScript object you constructed from the form fields to JSON before posting it to the page handler. Then you will convert the page handler method to return a `JsonResult` rather than a `ContentResult`.

When you specify `x-www-form-urlencoded` as the content type for a request, the framework knows it should parse the request body for the posted name-value pairs that constitute the data you are sending to the server. You let the framework know when you post JSON by setting the content type to `application/json`. But you also tell the page handler where to get the data. You do this by applying the `FromBody` attribute to the page handler parameter. Listing 11.15 shows how the page handler method looks with the changes needed to work with JSON. In the code, you create an anonymous type to represent the returned data. While this works fine for ad hoc situations, a more robust approach would involve declaring classes or records for your return types, so they are testable.

Listing 11.15 Using `JsonResult` and the `FromBody` attribute

```
public JsonResult OnPostBooking([FromBody]BookingInputModel model)    ❶
{
    var numberOfDays = (int)(model.EndDate.Value -
    ➥ model.StartDate.Value).TotalDays;
    var totalCost = numberOfDays * model.Property.DayRate *
```